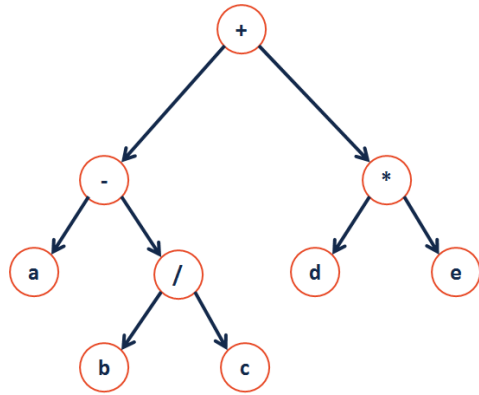


Traversals:

1. Pre-

2. In-

3. Post-

BinaryTree.cpp

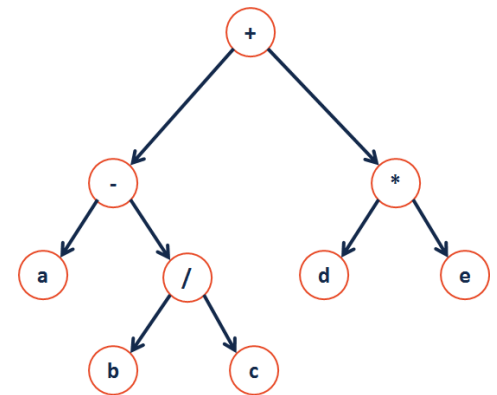
```
TreeNode * BinaryTree<T>::_copy(TreeNode * root) {
    if (root != NULL) {
        // ...
    }
}
```

BinaryTree.cpp

```
void BinaryTree<T>::_clear(TreeNode * root) {
    if (root != NULL) {
        // ...
    }
}
```

A Different Type of Traversal

Strategy:



BinaryTree.cpp

```
void BinaryTree<T>::levelOrder(TreeNode * root) {
```

Traversal vs. Search:

Breadth First Search:

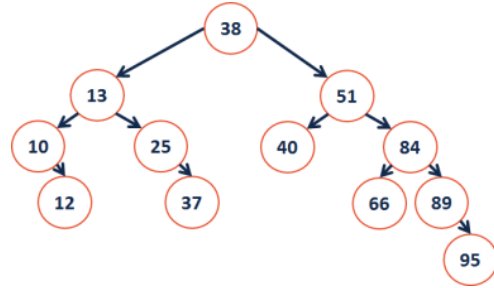
Depth First Search:

Dictionary ADT

Dictionary.h

```
1 #ifndef DICTIONARY_H
2 #define DICTIONARY_H
3
4
5 class Dictionary {
6     public:
7
8
9
10
11
12
13
14
15
16
17
18
19
20     private:
21
22
23 };
24 #endif
```

Binary Search Tree (BST)



Tree Analysis:

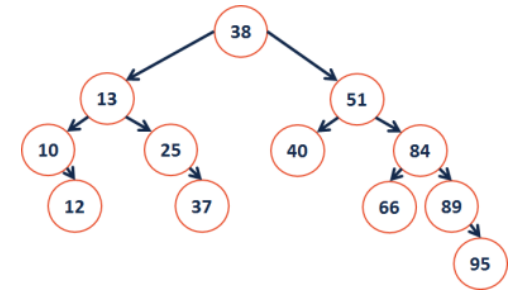
- Best case?
- Worst case?
- Insertion of a sorted list of elements?
- Running time?

BST.h

```

1  #ifndef DICTIONARY_H
2  #define DICTIONARY_H
3
4  template <class K, class V>
5  class BST {
6  public:
7      BST();
8      void insert(const K key, V value);
9      V remove(const K & key);
10     V find(const K & key) const;
11     TreeIterator traverse() const;
12 private:
13
14
15
16
17
18 };
19 #endif
  
```

Finding an element in a BST:



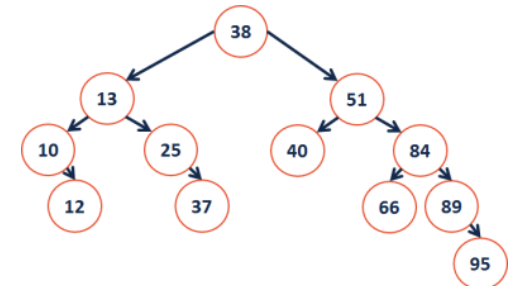
BST.cpp

```

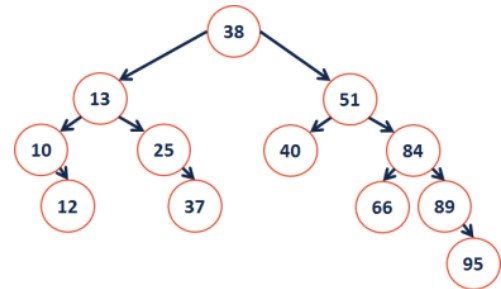
..._find
(TreeNode *& root, const K & key) const {
  }
  
```

...running time: _____

What if we did not pass a pointer by reference?



Inserting an element into a BST:



BST.cpp

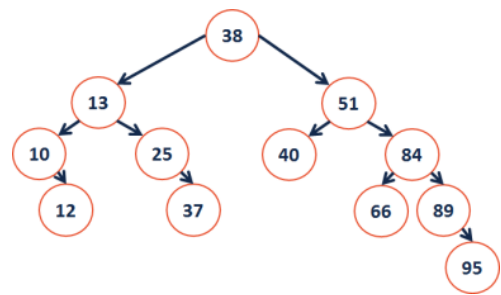
```
template <class K, class V>
void BST::_insert(TreeNode *& root, K key, V value) {

}
}
```

Running time? _____ Bound by? _____

Removing an element from a BST:

- _remove (40)
- _remove (25)
- _remove (10)
- _remove (13)



One-child Remove	Two-child remove

BinaryTree.cpp

```
template <class K, class V>
void BST::_remove(TreeNode *& root, const K & key) {

}
}
```

Running time? _____ Bound by? _____

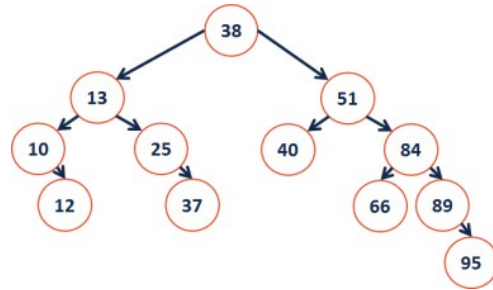
Removing an element from a BST:

_remove(40)

_remove(25)

_remove(10)

_remove(13)



```

template <class K, class V>
void BST::_remove(TreeNode *& root, const K & key) {

}
  
```

BST Analysis:

Every operation we have studied on a BST depends on:

...what is this in terms of the amount of data, **n**?

One-child Remove	Two-child remove

BST.cpp

Proving the relationship between **h** and **n**:

Q: What is the minimum number of nodes in tree of height **h**?

Q: What is the greatest possible height for a tree of **n** nodes?

Q: How many different ways are there to insert data into a BST?

Final BST Analysis:

For every height-based algorithm on a BST:

Lower Bound:

Upper Bound:

Why use this over a linked list?

Q: How does our data determine the height?

1 3 2 4 5 7 6 vs. 4 2 3 6 7 1 5

Q: What is the average height of every arrangement?

operation	BST Avg. Case	BST Worst Case	Sorted Array	Sorted List
find				
insert				
delete				
traverse				

CS 225 – Things To Be Doing:
1. MP3 is in Week 2; due on Monday 2. lab_trees is due on Sunday